Exercise answers

Part 1: exercises 1 to 7

Every answer below was run on Java 21 against the sample data; the comment under each one is its real output.

Exercise 1: filter

```
List<Transaction> failedOrPending = txns.stream()
        .filter(t -> "FAILED".equals(t.getStatus()) ||
"PENDING".equals(t.getStatus()))
        .collect(Collectors.toList());
// [T3, T5]
```

The OR has to sit inside one `filter`. Two filters in a row would mean FAILED and PENDING at the same time, which matches nothing.

Exercise 2: map

```
List<String> idAmounts = txns.stream()
        .filter(t -> "SUCCESS".equals(t.getStatus()))
        .map(t -> t.getId() + "=" + t.getAmount())
        .collect(Collectors.toList());
// [T1=500.0, T2=120.5, T4=900.0, T6=45.0]
```

Filter first, while the status is still available, then map.

Exercise 3: flatMap

```
List<String> debitTags = txns.stream()
        .filter(t -> "DEBIT".equals(t.getType()))
        .map(Transaction::getTags)
        .filter(Objects::nonNull)
        .flatMap(List::stream)
        .collect(Collectors.toList());
// [rent, monthly, bills, rent, monthly]
```

No DEBIT transaction here has null tags, but keep the null filter anyway; the next data set might have one.

Exercise 4: distinct

```
List<String> statuses = txns.stream()
        .map(Transaction::getStatus)
        .distinct()
        .collect(Collectors.toList());
// [SUCCESS, FAILED, PENDING]
```

Exercise 5: sorted

```
List<String> sortedIds = txns.stream()
        .sorted(Comparator.comparing(Transaction::getAccountId)

.thenComparing(Comparator.comparingDouble(Transaction::getAmount).reversed())
)
        .map(Transaction::getId)
        .collect(Collectors.toList());
// [T1, T2, T6, T4, T3, T5]
```

The `.reversed()` sits inside `thenComparing`, so only the amount is reversed.

Exercise 6: Comparator

```
List<String> byLengthDesc = names.stream()
        .sorted(Comparator.comparingInt(String::length).reversed()
                .thenComparing(Comparator.naturalOrder()))
        .collect(Collectors.toList());
// [Asha, Chen, Ben, Ben, Dev]
```

Here `.reversed()` comes before `thenComparing`, so it flips only the length rule.

Exercise 7: limit

```
List<Double> smallestDebits = txns.stream()
        .filter(t -> "DEBIT".equals(t.getType()))
        .map(Transaction::getAmount)
        .sorted()
        .limit(3)
        .collect(Collectors.toList());
// [45.0, 75.25, 120.5]
```

Sort first, then limit; the other way round would take the first three debits in list order.

Part 2: exercises 8 to 14

Exercise 8: count

```
long debitAccounts = txns.stream()
        .filter(t -> "DEBIT".equals(t.getType()))
        .map(Transaction::getAccountId)
        .distinct()
        .count();
// 3
```

Exercise 9: sum

```
double successfulCredits = txns.stream()
        .filter(t -> "CREDIT".equals(t.getType()))
        .filter(t -> "SUCCESS".equals(t.getStatus()))
        .mapToDouble(Transaction::getAmount)
        .sum();
// 1400.0   (T1 500.00 + T4 900.00)
```

Exercise 10: average

```
double averageDebit = txns.stream()
        .filter(t -> "DEBIT".equals(t.getType()))
        .mapToDouble(Transaction::getAmount)
        .average()
        .orElse(0.0);
// 135.1875   (540.75 / 4)
```

Exercise 11: max

```
String largestProblem = txns.stream()
        .filter(t -> "PENDING".equals(t.getStatus()) ||
"FAILED".equals(t.getStatus()))
        .max(Comparator.comparingDouble(Transaction::getAmount))
        .map(Transaction::getId)
        .orElse("NONE");
// T5   (300.00 beats T3's 75.25)
```

Exercise 12: findFirst

```
String acc3Status = txns.stream()
        .filter(t -> "ACC-3".equals(t.getAccountId()))
        .findFirst()
        .map(Transaction::getStatus)
        .orElse("NOT FOUND");
// PENDING
```

Exercise 13: allMatch

```
boolean acc1AllSuccess = txns.stream()
        .filter(t -> "ACC-1".equals(t.getAccountId()))
        .allMatch(t -> "SUCCESS".equals(t.getStatus()));
// true; the same check for ACC-2 gives false because of T3
```

An account with no transactions at all would also give true, so check for empty first when that matters.

Exercise 14: reduce

```
Optional<String> longest = names.stream()
        .reduce((a, b) -> b.length() > a.length() ? b : a);
// Optional[Asha]
```

Using `>` rather than `>=` keeps the earlier name on a tie, so Asha beats Chen.

Parts 3 to 5: exercises 15 to 21

Exercise 15: Optional

```
double firstFailedAmount = txns.stream()
        .filter(t -> "FAILED".equals(t.getStatus()))
        .findFirst()
        .map(Transaction::getAmount)
        .orElse(0.0);
// 75.25
```

Exercise 16: collect into a sorted set

```
Set<String> problemAccounts = txns.stream()
        .filter(t -> "FAILED".equals(t.getStatus()) ||
"PENDING".equals(t.getStatus()))
        .map(Transaction::getAccountId)
        .collect(Collectors.toCollection(TreeSet::new));
// [ACC-2, ACC-3]
```

Exercise 17: joining

```
String acc1Line = txns.stream()
        .filter(t -> "ACC-1".equals(t.getAccountId()))
        .map(Transaction::getId)
        .collect(Collectors.joining(", ", "ACC-1: ", ""));
// ACC-1: T1, T2, T6
```

The prefix supplies "ACC-1: " and the suffix is empty.

Exercise 18: toMap

```
Map<String, Double> largestByAccount = txns.stream()
        .collect(Collectors.toMap(
                Transaction::getAccountId,
                Transaction::getAmount,
                Double::max,            // two amounts for one account: keep
the bigger one
                TreeMap::new));
// {ACC-1=500.0, ACC-2=900.0, ACC-3=300.0}
```

Exercise 19: groupingBy

```
Map<String, Double> successTotals = txns.stream()
        .filter(t -> "SUCCESS".equals(t.getStatus()))
        .collect(Collectors.groupingBy(
                Transaction::getAccountId,
                TreeMap::new,
                Collectors.summingDouble(Transaction::getAmount)));
// {ACC-1=665.5, ACC-2=900.0}
```

ACC-3 is missing because its only transaction is PENDING. `successTotals.getOrDefault("ACC-3", 0.0)` safely returns 0.0.

Exercise 20: partitioningBy

```
Map<Boolean, Long> successSplit = txns.stream()
        .collect(Collectors.partitioningBy(
                t -> "SUCCESS".equals(t.getStatus()),
                Collectors.counting()));
// {false=2, true=4}
```

Exercise 21: streaming a map

```
List<String> over500 = totals.entrySet().stream()
        .filter(e -> e.getValue() > 500)
        .map(Map.Entry::getKey)
        .sorted()
        .collect(Collectors.toList());
// [ACC-1, ACC-2]
```